

Image Steganography Tool

Full Name: Dawson Erva

Email: dlerva@mtu.edu

Course: SAT 3310 – Scripting for Administration, Automation and Security

Date: March 20, 2026

1 Objective

The objective of this lab is to develop a security-focused script that utilizes image steganography to hide sensitive text data within a digital image file. The tool demonstrates how data can be concealed in plain sight, providing a layer of "security through obscurity" that is imperceptible to the human eye. The script is written in Python 3 and runs on both Windows and Linux platforms.

2 Introduction

Image steganography is the practice of concealing a file, message, or data within another image file. Unlike encryption, which scrambles the content of a message to make it unreadable, steganography hides the fact that a message is being sent at all. To an outside observer, the carrier image looks completely normal.

This script uses the Least Significant Bit (LSB) method to embed binary data into the pixels of a PNG image. Each pixel in an RGB image contains three color channels: red, green, and blue, each stored as a value from 0 to 255. By modifying only the last (least significant) bit of the blue channel, the color change is so minor – at most a difference of 1 out of 255 – that it is completely invisible to the human eye.

PNG is used as the file format because it is lossless, meaning no pixel data is altered during saving. JPEG compression would destroy the hidden bits, making recovery impossible.

3 Installation

To run this tool, the following environment must be configured:

3.1 Python 3.x

Ensure Python 3 is installed on your system. You can verify this by opening Command Prompt (Windows) or Terminal (Linux) and running:

```
Python  
python --version
```

If Python is not installed, download it from <https://python.org> and follow the installer. Make sure to check "Add Python to PATH" during installation on Windows.

3.2 Pillow Library

This script requires the Pillow module for pixel-level image manipulation. Install it using pip:

```
Python  
pip install Pillow
```

Verify the installation was successful by running:

```
Python  
python -c "from PIL import Image; print('Pillow OK')"
```

3.3 Recommended IDE

Any text editor or IDE can be used to view or edit the script. Visual Studio Code is recommended for its built-in Python support and terminal integration.

4 Configuration

Before executing the script, the following files must be present in the same folder:

- steganography_tool.py – the main script
- cover.png – any PNG image to act as the carrier for the hidden data. Larger images can hold larger messages. A standard 1920×1080 image can hold over 250,000 characters.
- secret.txt – a plain text file containing the message you wish to hide

The script also contains two configuration variables at the top that can be adjusted:

4.1 DEBUG Flag

Located at line 14 of the script. Set to True to print detailed step-by-step output for each pixel processed. Set to False for clean, minimal output. DEBUG mode is useful during testing but should be turned off for large images as it generates a large amount of terminal output.

Python

```
DEBUG = True    # Change to False to disable verbose output
```

4.2 DELIMITER

Located at line 18. This is the string appended to the end of your message before encoding so the decoder knows where the message ends. The default value is "#####". Do not change this unless you have a specific reason to, and ensure it matches in both encode and decode runs.

Python

```
DELIMITER = "#####"
```

5 Testing

Follow these steps to verify the tool works correctly. Screenshots should be taken at each step and inserted in the spaces indicated below.

5.1 Step 1 – Prepare Your Files

Place steganography_tool.py, cover.png, and secret.txt in the same folder. Open secret.txt and type a test message such as "This is a secret message for SAT 3310." Save and close the file.

[INSERT SCREENSHOT HERE – Your folder showing all three files: steganography_tool.py, cover.png, secret.txt]

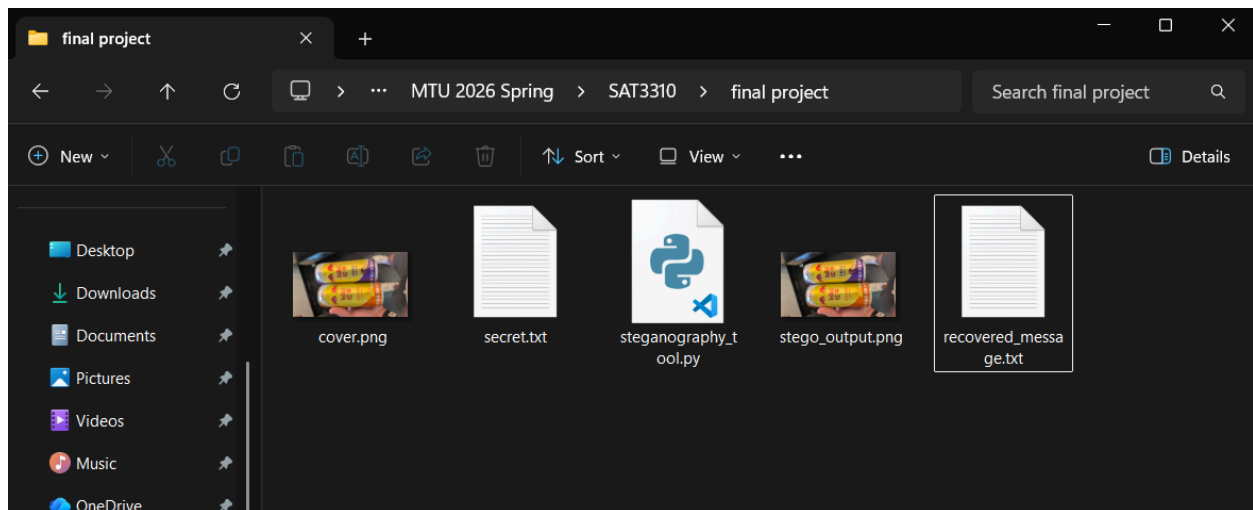


Figure 1 – Project folder with all required files present

5.2 Step 2 – Run the Script and Encode

Open Command Prompt, navigate to your project folder, and run the script:

```
Python  
python steganography_tool.py
```

The main menu will appear. Enter E to select Encode. The script will load secret.txt, load cover.png, and embed the message. A success message will confirm how many bits were embedded and that stego_output.png was created.

[INSERT SCREENSHOT HERE – Terminal showing the script menu and successful Encode output including bit count and output file confirmation]

```

Select an option:
[E] Encode - Hide a message in an image
[D] Decode - Extract a message from an image
[Q] Quit

Enter choice (E/D/Q): e

--- ENCODE MODE ---
[+] Secret message loaded from 'secret.txt' (161 characters)
[DEBUG] Message content: This is the secret message that we are hiding, the...
C:\Users\Home\MTU\MTU 2026 Spring\SAT3310\final project\steganography_tool.py:77: DeprecationWarning:
in Pillow 14 (2027-10-15). Use get_flattened_data instead.
  pixels = list(image.getdata())
[+] Cover image loaded: 'cover.png' (4080x2295, 9363600 pixels)
[DEBUG] Converted text to binary (1328 bits total)
[+] Message fits: 1328 bits needed, 9363600 pixels available
[DEBUG] Pixel 0: blue 103 -> 102 (embedded bit '0')
[DEBUG] Pixel 1: blue 103 -> 103 (embedded bit '1')
[DEBUG] Pixel 2: blue 103 -> 102 (embedded bit '0')
[DEBUG] Pixel 3: blue 103 -> 103 (embedded bit '1')
[DEBUG] Pixel 4: blue 103 -> 102 (embedded bit '0')
[DEBUG] Pixel 5: blue 103 -> 103 (embedded bit '1')
[DEBUG] Pixel 6: blue 103 -> 102 (embedded bit '0')
[DEBUG] Pixel 7: blue 103 -> 102 (embedded bit '0')
[DEBUG] Pixel 8: blue 101 -> 100 (embedded bit '0')

[SUCCESS] Message hidden successfully!
- Bits embedded : 1328
- Output file   : 'stego_output.png'
- Image size    : 4080x2295

```

Figure 2 – Successful encode output in the terminal

5.3 Step 3 – Compare the Images

Open both cover.png and stego_output.png side by side. Despite the hidden message being embedded, the two images should appear visually identical. This demonstrates the imperceptibility of the LSB method.

[INSERT SCREENSHOT HERE – Side-by-side comparison of cover.png (original) and stego_output.png (with hidden message) – they should look identical]



cover.png

stego_output.png

Figure 3 – Original cover image (left) and stego image (right) showing no visible difference

5.4 Step 4 – Decode the Message

Without closing the script, enter D to select Decode. The script will open stego_output.png, scan through the pixels extracting LSBs, and reconstruct the original message. The recovered text will be printed to the terminal and saved to recovered_message.txt.

[INSERT SCREENSHOT HERE – Terminal showing the Decode output with the recovered secret message printed]

```

Select an option:
[E] Encode - Hide a message in an image
[D] Decode - Extract a message from an image
[Q] Quit

Enter choice (E/D/Q): d

--- DECODE MODE ---
C:\Users\Home\MTU\MTU 2026 Spring\SAT3310\final project\steganography_tool.py:136
in Pillow 14 (2027-10-15). Use get_flattened_data instead.
pixels = list(image.getdata())
[+] Stego image loaded: 'stego_output.png' (4080x2295)
[DEBUG] Pixel 0: blue=102, LSB=0
[DEBUG] Pixel 1: blue=103, LSB=1
[DEBUG] Pixel 2: blue=102, LSB=0
[DEBUG] Pixel 3: blue=103, LSB=1
[DEBUG] Pixel 4: blue=102, LSB=0
[DEBUG] Pixel 1321: blue=66, LSB=0
[DEBUG] Pixel 1322: blue=69, LSB=1
[DEBUG] Pixel 1323: blue=66, LSB=0
[DEBUG] Pixel 1324: blue=62, LSB=0
[DEBUG] Pixel 1325: blue=60, LSB=0
[DEBUG] Pixel 1326: blue=59, LSB=1
[DEBUG] Pixel 1327: blue=59, LSB=1

[SUCCESS] Hidden message recovered!
- Characters extracted : 161

--- SECRET MESSAGE ---
This is the secret message that we are hiding, there is some special message here !!

YEP.

the end.
-----

[+] Message also saved to 'recovered_message.txt'

Select an option:
[E] Encode - Hide a message in an image
[D] Decode - Extract a message from an image
[Q] Quit

Enter choice (E/D/Q): |

```

Figure 4 – Successful decode output showing the recovered message

5.5 Step 5 – Confirm recovered_message.txt

Open recovered_message.txt in your project folder and confirm it contains the exact text from your original secret.txt. This confirms full end-to-end data integrity.

[INSERT SCREENSHOT HERE – recovered_message.txt open in Notepad showing the recovered text matches the original secret.txt]

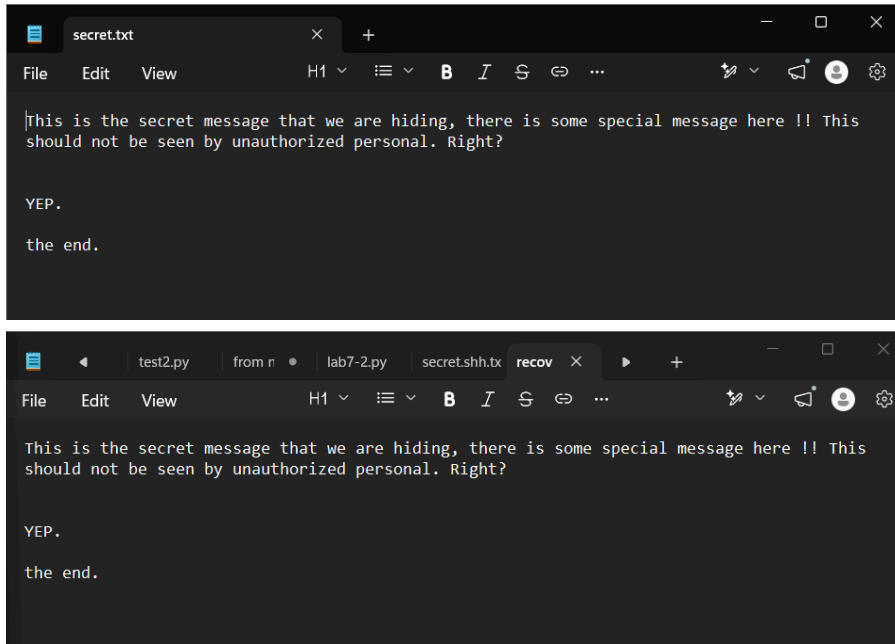


Figure 5 – Recovered message file confirming successful extraction

6 Troubleshooting

6.1 ModuleNotFoundError: No module named 'PIL'

Pillow is not installed, or was installed in a different Python environment. Run the following command and try again:

```
Python  
pip install Pillow
```

If you have multiple Python versions installed, try:

```
Python  
python -m pip install Pillow
```

6.2 [ERROR] 'cover.png' not found

The cover image is either missing or not named correctly. Ensure the file is named exactly cover.png (all lowercase) and is in the same folder as the script. You can save any image as a PNG from Paint by going to File > Save As > PNG picture.


```
--- ENCODE MODE ---  
[+] Secret message loaded from 'secret.txt' (161 characters)  
[DEBUG] Message content: This is the secret message that we are hiding, the...  
[ERROR] 'cover.png' not found. Please provide a PNG image.
```

6.3 [ERROR] 'secret.txt' not found

The secret message file is missing. Create a plain text file named secret.txt in the same folder as the script and type your message inside it.

```
--- ENCODE MODE ---  
[ERROR] 'secret.txt' not found. Please create it with your secret message.
```

6.4 [ERROR] Message too large

The message in secret.txt is larger than the carrier image can hold. Each pixel can store one bit, so the image needs at least as many pixels as bits in your message. Switch to a larger image (a standard 1920×1080 PNG can hold over 250,000 characters).

6.5 [ERROR] Delimiter not found during Decode

This means the image being decoded does not contain a hidden message, or it was saved in a lossy format (such as JPEG) after encoding, destroying the hidden bits. Always use PNG files throughout the entire process.

6.6 Permission Denied

The script cannot read or write a file because it is open in another application. Close any open instances of cover.png, stego_output.png, secret.txt, or recovered_message.txt and try again.

7 References

- Michigan Technological University SAT 3310 Course Project Guidelines
- Pillow (PIL Fork) Documentation: <https://python-pillow.github.io/>
- Least Significant Bit Steganography – general cryptography and security coursework

8 Script

The complete Python script is included below. Copy and paste this into a file named steganography_tool.py to run it.

Python

```
# =====
# Image Steganography Tool
# Author: Dawson Erva
# Course: SAT 3310 - Scripting for Administration, Automation and Security
# Description: Hides and retrieves text from a file inside a PNG image
#               using the Least Significant Bit (LSB) method.
# Note: the image file and text file should be in the same folder as this
#       script at run time. Image file should be named "cover.png" and the
#       text file should be named : "secret.txt"
# =====

from PIL import Image
import os
import sys

# --- Debug Flag ---
# Set to True to print step-by-step progress, False for clean output
DEBUG = True

# --- Delimiter ---
# This unique string marks the end of the hidden message during decoding
DELIMITER = "#####"

def debug_print(message):
    """Prints a message only if DEBUG mode is enabled."""
    if DEBUG:
        print(f" [DEBUG] {message}")

def text_to_binary(text):
    """Converts a string of text into a binary string."""
    binary = ""
    for char in text:
        # Convert each character to its 8-bit binary representation
        binary += format(ord(char), "08b")
    debug_print(f"Converted text to binary ({len(binary)} bits total)")
    return binary

def binary_to_text(binary):
    """Converts a binary string back into readable text."""
    text = ""
    # Process 8 bits at a time (one character)
    for i in range(0, len(binary), 8):
```

```

        byte = binary[i:i + 8]
        if len(byte) == 8:
            text += chr(int(byte, 2))
    return text

# =====
# ENCODE FUNCTION
# Reads a secret message from secret.txt and hides it inside cover.png
# =====
def encode_message():
    print("\n--- ENCODE MODE ---")

    # --- File I/O: Read the secret message ---
    secret_file = "secret.txt"
    if not os.path.exists(secret_file):
        print(f"[ERROR] '{secret_file}' not found. Please create it with your secret message.")
        return

    with open(secret_file, "r") as f:
        message = f.read()
        print(f"[+] Secret message loaded from '{secret_file}' ({len(message)} characters)")
        debug_print(f"Message content: {message[:50]}{'...' if len(message) > 50 else ''}")

    # --- File I/O: Open the cover image ---
    cover_file = "cover.png"
    if not os.path.exists(cover_file):
        print(f"[ERROR] '{cover_file}' not found. Please provide a PNG image.")
        return

    image = Image.open(cover_file)
    # Ensure image is in RGB mode for consistent pixel access
    image = image.convert("RGB")
    pixels = list(image.getdata())
    print(f"[+] Cover image loaded: '{cover_file}' ({image.width}x{image.height}, {len(pixels)} pixels)")

    # --- Convert message + delimiter to binary ---
    full_message = message + DELIMITER
    binary_message = text_to_binary(full_message)
    total_bits = len(binary_message)

    # --- Condition Check: Does the image have enough pixels to hold the message?
    ---
    if total_bits > len(pixels):
        print(f"[ERROR] Message too large! Need {total_bits} pixels, image only has {len(pixels)}.")
        return

```

```

    print(f"[+] Message fits: {total_bits} bits needed, {len(pixels)} pixels
available")

# --- Loop: Embed each bit into the LSB of the blue channel ---
encoded_pixels = []
bit_index = 0

for i, pixel in enumerate(pixels):
    r, g, b = pixel

    # Condition check: If there are still bits to embed, modify the LSB
    if bit_index < total_bits:
        # Clear the last bit of blue, then set it to the message bit
        new_b = (b & 0b1111110) | int(binary_message[bit_index])
        encoded_pixels.append((r, g, new_b))
        bit_index += 1
        debug_print(f"Pixel {i}: blue {b} -> {new_b} (embedded bit
'{binary_message[bit_index-1]}')")
    else:
        # No more bits to embed; keep pixel unchanged
        encoded_pixels.append((r, g, b))

# --- File I/O: Save the stego image ---
output_file = "stego_output.png"
stego_image = Image.new("RGB", image.size)
stego_image.putdata(encoded_pixels)
stego_image.save(output_file)

# --- Display: Confirm success ---
print(f"\n[SUCCESS] Message hidden successfully!")
print(f" - Bits embedded : {bit_index}")
print(f" - Output file   : '{output_file}'")
print(f" - Image size      : {image.width}x{image.height}")

# =====
# DECODE FUNCTION
# Extracts a hidden message from stego_output.png
# =====
def decode_message():
    print("\n--- DECODE MODE ---")

    # --- File I/O: Open the stego image ---
    stego_file = "stego_output.png"
    if not os.path.exists(stego_file):
        print(f"[ERROR] '{stego_file}' not found. Run Encode first.")
        return

    image = Image.open(stego_file).convert("RGB")
    pixels = list(image.getdata())

```

```

    print(f"[+] Stego image loaded: '{stego_file}'
    ({image.width}x{image.height})")

    # --- Loop: Extract the LSB from the blue channel of each pixel ---
    binary_message = ""
    for i, pixel in enumerate(pixels):
        r, g, b = pixel
        # Extract the least significant bit of the blue channel
        lsb = b & 1
        binary_message += str(lsb)
        debug_print(f"Pixel {i}: blue={b}, LSB={lsb}")

    # --- Condition Check: Every 8 bits, check for the delimiter ---
    if len(binary_message) % 8 == 0:
        # Decode what we have so far and check if delimiter has appeared
        current_text = binary_to_text(binary_message)
        if DELIMITER in current_text:
            # Stop - we have the full message
            recovered = current_text.split(DELIMITER)[0]
            break
    else:
        # Loop finished without finding the delimiter
        print("[ERROR] Delimiter not found. Image may not contain a hidden
message.")
        return

    # --- Display: Print the recovered secret message ---
    print(f"\n[SUCCESS] Hidden message recovered!")
    print(f" - Characters extracted : {len(recovered)}")
    print(f"\n--- SECRET MESSAGE ---")
    print(recovered)
    print(f"-----")

    # --- File I/O: Save the recovered message to a file ---
    output_file = "recovered_message.txt"
    with open(output_file, "w") as f:
        f.write(recovered)
    print(f"\n[+] Message also saved to '{output_file}'")

# =====
# MAIN - Menu / Entry Point
# =====
def main():
    print("=" * 45)
    print("  Image Steganography Tool - SAT 3310")
    print("=" * 45)
    print(f"  Debug mode: {'ON' if DEBUG else 'OFF'}")
    print("=" * 45)

    # Loop: Keep showing the menu until the user quits

```

```

while True:
    print("\nSelect an option:")
    print("  [E] Encode - Hide a message in an image")
    print("  [D] Decode - Extract a message from an image")
    print("  [Q] Quit")

    # Input from user
    choice = input("\nEnter choice (E/D/Q): ").strip().upper()

    # Condition check: Route to the correct function
    if choice == "E":
        encode_message()
    elif choice == "D":
        decode_message()
    elif choice == "Q":
        print("\n[+] Exiting. Goodbye!")
        sys.exit(0)
    else:
        print("[!] Invalid choice. Please enter E, D, or Q.")

# --- Entry point ---
if __name__ == "__main__":
    main()

```